



Balancer ReCLAMM

Security Review

Cantina Managed review by:
Phaze, Lead Security Researcher
Cryptara, Security Researcher

April 20, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
2.1	Scope	3
3	Findings	4
3.1	Low Risk	4
3.1.1	Out-of-range repricing enables a profitable round trip against LPs on thinly traded pairs	4
3.1.2	Initialization may not enforce the documented centeredness-above-margin requirement	9
3.1.3	Recovery Mode Withdrawals Permanently Inflate Virtual Balances; Documented Self-Correction Paths Are Incorrect	11
3.2	Informational	13
3.2.1	Price-ratio update setters are inconsistent with the lock model used by comparable admin setters	13
3.2.2	Long-idle virtual-balance decay can brick a ReClamm pool after near depletion	13
3.2.3	Uninitialized price-ratio getters revert on deployed but uninitialized pools	15
3.2.4	Centeredness boundary is described inconsistently across docs and implementation	16
3.2.5	Factory accepts invalid initial dynamic params that cause initialization to revert	16
3.2.6	Target price equal to the upper bound can leave a deployed pool non-initializable . .	18
3.2.7	Price-ratio update speed check uses a linear approximation	18
3.2.8	Reverting Rate Provider Traps Pool in Recovery Mode Indefinitely	19
3.2.9	toDailyPriceShiftBase Silently Disables Price-Shifting for Sub-Threshold Inputs; NatSpec Formula Inverted	21
3.2.10	Per-Block Virtual Balance Freeze Creates First-Mover MEV Surface	22
3.2.11	Unused _MAX_TOKEN_DECIMALS constant	24
3.2.12	PriceRatioDeltaBelowMin Error Parameter and NatSpec Use Wrong Domain Label	24
3.2.13	Centeredness-preserving price-ratio updates can shift spot price in edge cases	25

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

From Apr 6th to Apr 13th the Cantina team conducted a review of [reclamm](#) on commit hash [a55a5770](#). The team identified a total of **16** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	3	3	0
Gas Optimizations	0	0	0
Informational	13	11	2
Total	16	14	2

2.1 Scope

The security review had the following components in scope for [reclamm](#) on commit hash [a55a5770](#):

```
contracts
├── ReClammPool.sol
├── ReClammPoolFactory.sol
├── ReClammPoolHelper.sol
└── lib
    ├── ReClammMath.sol
    └── ReClammPoolFactoryLib.sol
```

3 Findings

3.1 Low Risk

3.1.1 Out-of-range repricing enables a profitable round trip against LPs on thinly traded pairs

Severity: Low Risk

Context: [ReClammPool.sol#L190-L222](#)

Summary: ReCLAMM keeps adjusting its quoted range after the pool drifts past the centeredness margin. If the pool stays out of range, the same actor that pushed it there can unwind against a later curve and extract value from LP balances.

With the repository's 100% test-default price shift exponent, this can become profitable after only a few seconds. With a production-style 5% exponent and the current 0.001% ReCLAMM minimum swap fee, the same path still exists, but the attacker needs roughly 47-50 seconds of idle out-of-range time. That turns the issue into a multi-block execution risk: the attacker must fund the initial bad trade, wait several blocks, avoid being arb'd, and win the unwind.

Description: ReCLAMM is not a static concentrated-liquidity AMM. Once centeredness falls below the configured margin, `computeCurrentVirtualBalances()` moves the virtual balances over time so the pool follows the observed market price instead of sitting in a stale range. That design removes manual repositioning, but it also means an idle out-of-range pool quotes a different curve later.

If an actor pushes the pool past the centeredness margin and no one trades after them, the imbalanced side's virtual balance decays. The same actor can later unwind and recover the token spent on the first leg while keeping surplus of the other token. The pool does not need to be fully drained; a full drain is just the most visible version of the same path.

The preconditions for this to matter:

1. The pool has to be pushed past the centeredness margin and into a meaningfully out-of-range state.
2. It has to stay there long enough for the automatic repricing to move things by a useful amount.
3. Outside arbitrage and ordinary flow have to be quiet during that window, so the same actor is the one who closes the imbalance.

The same-block guard is important: if `lastTimestamp == block.timestamp`, virtual balances are returned unchanged, so a same-block round trip cannot use repricing. Past that guard, exploitability is mostly about elapsed time, swap fee, and whether another trader closes the imbalance first.

Under the 100% test-default shift exponent, the pool first falls out of range at about 45% token A removed. The round trip becomes profitable after about 3 seconds. Under a 5% shift exponent and the current 0.001% minimum swap fee, the first profitable delay was 47 seconds for a 45% token A removal and 50 seconds for a full removal. On a 12 second block-time chain, this is not a next-block attack. It requires several blocks of idleness and a successful unwind before competing arbitrage.

The documentation already warns operators away from thin, low-liquidity, or manipulable markets. This finding names the concrete LP-loss path behind that warning.

Impact Explanation: When the preconditions hold, an actor can take value directly from LP balances without any external price move. The attacker pays for the first price-impacting trade, waits for out-of-range repricing, then unwinds against the later curve.

At 5% shift and 0.001% swap fee, the first profitable points are barely profitable. In the local full-removal run, the 50 second case produced only 0.000056222171614202 token A of profit while requiring the attacker to fund the full opening leg. Larger idle windows increase the profit, but also increase the chance that a searcher or normal trader closes the imbalance first.

Likelihood Explanation: Likelihood is low on deep, monitored pairs because the attacker has to keep the pool out of range for multiple blocks and then win the unwind. If another trader corrects the pool first, the attacker is left with the intentionally bad opening trade and fees.

The scenario becomes more realistic on thin, quiet, newly deployed, poorly monitored, or manipulable pairs. The capital requirement is not a hard defense; it mostly scales the attacker's exposure and makes the waiting period more expensive to carry.

Proof of Concept: The harness below pins down where repricing first engages and how quickly the round trip becomes profitable. It runs against a pool deployed with the default test parameters, and follow-up local variants set the shift exponent to 5%.

Selected results from the sweep:

- 44% token A out: pool stays in range, repricing never starts, no profit available.
- 45% token A out: pool crosses the margin; round trip profitable after about 3 seconds.
- 99% token A out: profitable after about 12 seconds without taking token A to zero.
- 100% token A out: virtual balances reprice after 1 second, but the round trip still needs a few seconds to clear fees.

Follow-up results with the daily price shift exponent set to 5%:

- At the current ReCLAMM minimum swap fee of 0.001%, 44% token A out stayed in range and did not profit in the two-minute sweep.
- At 0.001%, 45% token A out first became profitable at 47 seconds.
- At 0.001%, 100% token A out first became profitable at 50 seconds.
- At a hypothetical 0.0006% swap fee, 100% token A out first became profitable at 30 seconds.
- At a hypothetical 0.0005% swap fee, 100% token A out first became profitable at 25 seconds.

The 5%/0.001% full-removal case is not profitable in the next normal block. It requires elapsed timestamp time close to 50 seconds, so the relevant practical risk is a multi-block idle window where the attacker can keep other flow from closing the imbalance and can win the unwind transaction.

Proof of Concept: test/foundry/ReClammRoundTripDesignRisk.t.sol:

```
// SPDX-License-Identifier: GPL-3.0-or-later

pragma solidity ^0.8.24;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { ReClammPool } from "../../contracts/ReClammPool.sol";
import { a, b } from "../../contracts/lib/ReClammMath.sol";
import { BaseReClammTest } from "../utils/BaseReClammTest.sol";

contract ReClammRoundTripDesignRiskTest is BaseReClammTest {
    struct RoundTripResult {
        uint256 amountAReceived;
        uint256 amountBSpent;
        uint256 amountASpentToRecoverB;
        uint256 attackerProfitTokenA;
        uint256 attackerProfitTokenB;
        uint256 centerednessBefore;
        uint256 centerednessAfterFirstSwap;
        uint256 centerednessBeforeSecondSwap;
        bool virtualBalancesChanged;
    }

    function testRoundTripRequiresPositiveTimeElapsed() public {
        (IERC20 tokenA, IERC20 tokenB) = _getPoolTokens();
        uint256 initialPoolBalanceA = _getPoolRawBalance(a);

        RoundTripResult memory sameTimestamp = _executeRoundTrip(
            tokenA,
            tokenB,
            initialPoolBalanceA,
            0,
            false
        );

        assertFalse(sameTimestamp.virtualBalancesChanged, "virtual balances unexpectedly
        ↪ changed without time passing");
    }
}
```

```

assertEq(sameTimestamp.attackerProfitTokenA, 0, "same-block round trip should not
↳ profit in token A");
assertEq(sameTimestamp.attackerProfitTokenB, 0, "same-block round trip should not
↳ profit in token B");
assertGt(
    sameTimestamp.amountASpentToRecoverB,
    sameTimestamp.amountAReceived,
    "same-block round trip should lose on the unchanged curve"
);
}

function testRoundTripNotYetProfitableAfterOneSecondAtFullDrain() public {
    (IERC20 tokenA, IERC20 tokenB) = _getPoolTokens();
    uint256 initialPoolBalanceA = _getPoolRawBalance(a);

    RoundTripResult memory result = _executeRoundTrip(tokenA, tokenB,
↳ initialPoolBalanceA, 1, true);

    assertTrue(result.virtualBalancesChanged, "virtual balances did not change after
↳ one second");
    assertEq(result.attackerProfitTokenA, 0, "one-second round trip unexpectedly
↳ profits in token A");
}

function testRoundTripCanProfitWithoutDrainingPool() public {
    (IERC20 tokenA, IERC20 tokenB) = _getPoolTokens();
    uint256 initialPoolBalanceA = _getPoolRawBalance(a);

    // Leave 1% of token A in the pool.
    uint256 amountAOut = (initialPoolBalanceA * 99) / 100;

    RoundTripResult memory result = _executeRoundTrip(tokenA, tokenB, amountAOut,
↳ 12, true);

    assertGt(_getPoolRawBalance(a), 0, "pool was unexpectedly drained");
    assertTrue(result.virtualBalancesChanged, "virtual balances did not change");
    assertGt(result.attackerProfitTokenA, 0, "partial-drain round trip did not profit
↳ in token A");
}

function testEmitRoundTripProfitMatrix() public {
    (IERC20 tokenA, IERC20 tokenB) = _getPoolTokens();
    uint256 initialPoolBalanceA = _getPoolRawBalance(a);

    uint256[11] memory outPercents = [uint256(44), 45, 50, 60, 70, 80, 85, 90, 95,
↳ 99, 100];
    uint256[9] memory delays = [uint256(0), 1, 2, 3, 5, 10, 11, 12, 60];

    for (uint256 i = 0; i < outPercents.length; ++i) {
        for (uint256 j = 0; j < delays.length; ++j) {
            uint256 amountAOut = outPercents[i] == 100
                ? initialPoolBalanceA
                : (initialPoolBalanceA * outPercents[i]) / 100;

            uint256 snapshotId = vm.snapshotState();
            RoundTripResult memory result = _executeRoundTrip(tokenA, tokenB,
↳ amountAOut, delays[j], true);

            emit log_named_uint("out_percent", outPercents[i]);
            emit log_named_uint("delay_seconds", delays[j]);
            emit log_named_uint("amount_a_out", amountAOut);
            emit log_named_uint("centeredness_after_first_swap",
↳ result.centerednessAfterFirstSwap);
            emit log_named_uint("centeredness_before_second_swap",
↳ result.centerednessBeforeSecondSwap);
            emit log_named_uint("amount_b_spent", result.amountBSpent);

```

```

        emit log_named_uint("amount_a_spent_to_recover_b",
            ↪ result.amountASpentToRecoverB);
        emit log_named_uint("attacker_profit_token_a",
            ↪ result.attackerProfitTokenA);
        emit log_named_uint("attacker_profit_token_b",
            ↪ result.attackerProfitTokenB);

        vm.revertToState(snapshotId);
    }
}

function testEmitMarginCrossingThreshold() public {
    (IERC20 tokenA, IERC20 tokenB) = _getPoolTokens();
    uint256 initialPoolBalanceA = _getPoolRawBalance(a);
    uint256[8] memory outPercents = [uint256(35), 40, 45, 46, 47, 48, 49, 50];

    for (uint256 i = 0; i < outPercents.length; ++i) {
        uint256 snapshotId = vm.snapshotState();
        uint256 amountAOut = (initialPoolBalanceA * outPercents[i]) / 100;

        vm.prank(alice);
        router.swapSingleTokenExactOut(
            pool,
            tokenB,
            tokenA,
            amountAOut,
            MAX_UINT256,
            MAX_UINT256,
            false,
            bytes("")
        );

        (uint256 centerednessAfterFirstSwap, ) =
            ↪ ReClammPool(pool).computeCurrentPoolCenteredness();

        emit log_named_uint("out_percent", outPercents[i]);
        emit log_named_uint("amount_a_out", amountAOut);
        emit log_named_uint("centeredness_after_first_swap",
            ↪ centerednessAfterFirstSwap);
        emit log_named_uint("margin", ReClammPool(pool).getCenterednessMargin());
        emit log_named_uint("is_within_target_range",
            ↪ ReClammPool(pool).isPoolWithinTargetRange() ? 1 : 0);

        vm.revertToState(snapshotId);
    }
}

function _executeRoundTrip(
    IERC20 tokenA,
    IERC20 tokenB,
    uint256 amountAOut,
    uint256 delay,
    bool advanceTimestamp
) internal returns (RoundTripResult memory result) {
    uint256 attackerTokenABefore = tokenA.balanceOf(alice);
    uint256 attackerTokenBBefore = tokenB.balanceOf(alice);

    (result.centerednessBefore, ) =
        ↪ ReClammPool(pool).computeCurrentPoolCenteredness();

    vm.prank(alice);
    result.amountBSpent = router.swapSingleTokenExactOut(
        pool,
        tokenB,
        tokenA,

```

```

        amountAOut,
        MAX_UINT256,
        MAX_UINT256,
        false,
        bytes("")
    );

    result.amountAReceived = tokenA.balanceOf(alice) - attackerTokenABefore;
    (result.centerednessAfterFirstSwap, ) =
    ↪ ReClammPool(pool).computeCurrentPoolCenteredness();

    if (advanceTimestamp && delay > 0) {
        skip(delay);
    }

    (, , result.virtualBalancesChanged) =
    ↪ ReClammPool(pool).computeCurrentVirtualBalances();
    (result.centerednessBeforeSecondSwap, ) =
    ↪ ReClammPool(pool).computeCurrentPoolCenteredness();

    vm.prank(alice);
    result.amountASpentToRecoverB = router.swapSingleTokenExactOut(
        pool,
        tokenA,
        tokenB,
        result.amountBSpent,
        MAX_UINT256,
        MAX_UINT256,
        false,
        bytes("")
    );

    uint256 attackerTokenAAfter = tokenA.balanceOf(alice);
    uint256 attackerTokenBAfter = tokenB.balanceOf(alice);

    if (attackerTokenAAfter > attackerTokenABefore) {
        result.attackerProfitTokenA = attackerTokenAAfter - attackerTokenABefore;
    }

    if (attackerTokenBAfter > attackerTokenBBefore) {
        result.attackerProfitTokenB = attackerTokenBAfter - attackerTokenBBefore;
    }
}

function _getPoolTokens() internal view returns (IERC20 tokenA, IERC20 tokenB) {
    (IERC20[] memory tokens, , , ) = vault.getPoolTokenInfo(pool);
    tokenA = tokens[a];
    tokenB = tokens[b];
}

function _getPoolRawBalance(uint256 tokenIndex) internal view returns (uint256) {
    (, , uint256[] memory balances, ) = vault.getPoolTokenInfo(pool);
    return balances[tokenIndex];
}
}

```

`testRoundTripRequiresPositiveTimeElapsed` confirms the same-block lower bound: with no time passing between the two swaps, the virtual balances do not change and the attacker loses on fees. `testRoundTripCanProfitWithoutDrainingPool` shows the partial-drain case under the 100% test-default shift exponent: leaving 1% of token A in the pool and waiting 12 seconds is already enough to extract token A profit straight out of LP balances. `testEmitMarginCrossingThreshold` and `testEmitRoundTripProfitMatrix` produce the default-parameter sweep summarised above. A local follow-up variant set the shift exponent to 5% and swept the first profitable delay at 0.001%, 0.0006%, and 0.0005% swap fees, producing the production-rate results listed above.

Recommendation: The existing deployment guidance already lands on the right conclusion. The suggestion here is to name the failure mode it is protecting against, so an integrator reading the docs can see why “deep liquidity, non-manipulable prices” is a hard requirement and not a soft preference.

One option is to add a short paragraph next to the existing deep-liquidity guidance, along the lines of:

On thinly traded or poorly arbitrated pairs, an actor that pushes the pool past the centeredness margin and waits for the automatic repricing to take effect can later unwind against a different curve and take value directly from LP balances. The repricing assumes that meaningful imbalances are closed quickly by competing flow; when that assumption does not hold, the same mechanism works against LPs.

The current fee settings already create a practical minimum delay before the round trip can cross breakeven at production-style shift rates. That delay is still an economic side effect rather than an explicit system invariant. If the team wants a stronger protocol-level guardrail, the automatic out-of-range price update could require the pool to remain outside the centeredness margin for a configured block-delay period before virtual-balance decay begins. For example, requiring the pool to stay out of range for 10 blocks before range tracking starts would give arbitrageurs and ordinary flow a deterministic chance to close the imbalance, and would make the attacker's required block-control window much longer than the current fee-derived breakeven window.

This should be treated as a design tradeoff rather than a free fix. A delay would make the pool less responsive after a real market move, so it may reduce one of ReCLAMM's intended benefits. But it would make the LP-loss path harder to rely on by turning the current practical delay into an explicit requirement.

The existing test suite already exercises out-of-range repricing in several places (`testOutOfRangeSwapExactIn__Fuzz`, `testOutOfRangePriceRatioUpdatingSwapExactIn__Fuzz`, `testOutOfRangeSwapExactOut__Fuzz`, `testOutOfRangeBeforeSetCenterednessMargin`, `testAddLiquidityOutOfRange__Fuzz`, and so on), but these tests only check that `lastVirtualBalances` updates correctly after a single swap. None of them assert anything about the round-trip path where the same actor opens the imbalance, waits for the repricing, and closes it again. It is worth landing the harness above alongside those existing cases as a known scenario, both to make the team's awareness of the tradeoff visible and to give a regression anchor if the repricing rule is ever changed.

Balancer: Fixed in [PR 196](#).

Cantina Managed: Fix verified.

3.1.2 Initialization may not enforce the documented centeredness-above-margin requirement

Severity: Low Risk

Context: [ReClammPool.sol#L282-L305](#)

Description: The docs state that initialization should fail if the initial centeredness is below the configured margin. The current initialization path appears to check balance ratio and price consistency, then store the initial virtual balances and dynamic parameters without separately checking that centeredness is already above the configured margin.

This does not appear to trap LP funds or block proportional exit. The practical effect is narrower. A pool may initialize successfully in a state the docs describe as invalid, start out of range immediately, and become eligible for repricing from the first later-block review. This makes the issue primarily about deployment safety and operator expectations.

It is also worth separating a theoretical parameter check from a real initialization check. For non-rate pools, the centeredness condition can likely be screened earlier from the configured initial prices alone. For pools with rate providers, the effective initialization prices are adjusted using live rates at initialization time, so only the initialization path can reliably enforce that the actual pool starts in range.

Proof of Concept: `test/poc/ReClammInitializationCenterednessGap.t.sol`:

```
// SPDX-License-Identifier: GPL-3.0-or-later

pragma solidity ^0.8.24;
```

```

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { ArrayHelpers } from
↳ "@balancer-labs/v3-solidity-utils/contracts/test/ArrayHelpers.sol";

import { BaseReClammTest } from "../foundry/utils/BaseReClammTest.sol";
import { ReClammPool } from "../../contracts/ReClammPool.sol";
import { a, b } from "../../contracts/lib/ReClammMath.sol";

contract ReClammInitializationCenterednessGapPoC is BaseReClammTest {
    using ArrayHelpers for *;

    function testInitializeSucceedsEvenWhenInitialCenterednessIsBelowMargin() public {
        setInitializationPrices(1000e18, 4000e18, 1200e18);

        (address newPool, ) = _createPool([address(usdc), address(dai)].toMemoryArray(),
        ↳ "InitCenterednessGapPoC");
        IERC20[] memory poolTokens = vault.getPoolTokens(newPool);

        uint256[] memory initialBalances =
        ↳ ReClammPool(newPool).computeInitialBalancesRaw(poolTokens[a], 1_000e18);

        vm.startPrank(lp);
        router.initialize(newPool, poolTokens, initialBalances, 0, false, bytes(""));
        vm.stopPrank();

        assertTrue(vault.isPoolInitialized(newPool), "Pool initialization should
        ↳ succeed");

        uint256 margin = ReClammPool(newPool).getCenterednessMargin();
        (uint256 centeredness, ) = ReClammPool(newPool).computeCurrentPoolCenteredness();

        emit log_named_uint("initial_balance_a", initialBalances[a]);
        emit log_named_uint("initial_balance_b", initialBalances[b]);
        emit log_named_uint("centeredness_margin", margin);
        emit log_named_uint("initial_centeredness", centeredness);

        assertLt(centeredness, margin, "Pool should initialize below the configured
        ↳ margin");
        assertFalse(ReClammPool(newPool).isPoolWithinTargetRange(), "Pool should start
        ↳ out of range");

        skip(1);

        (, , bool changed) = ReClammPool(newPool).computeCurrentVirtualBalances();

        assertTrue(changed, "Pool should begin repricing on the next block");
    }
}

```

Command:

```

forge test --match-path test/poc/ReClammInitializationCenterednessGap.t.sol --match-test
↳ testInitializeSucceedsEvenWhenInitialCenterednessIsBelowMargin -vvvv

```

Observed result:

- Initialization succeeds.
- centeredness_margin = 2000000000000000000.
- initial_centeredness = 115587109997047935.
- isPoolWithinTargetRange() returns false immediately after initialization.
- computeCurrentVirtualBalances() returns changed = true one block later.

Recommendation: Consider enforcing the documented centeredness-above-margin condition during

initialization.

One approach could be to add a lightweight factory-time precheck for non-rate pools, where the initial centeredness can be derived entirely from the configured parameters. Even if that is added, the initialization path still appears to be the right place for the final invariant check, since that is where the actual balances and any live rate adjustments are known.

If the current behavior is intentional, consider updating the docs so they describe the real initialization guarantees more precisely.

Balancer: Fixed in [PR 180](#).

Cantina Managed: Fix verified.

3.1.3 Recovery Mode Withdrawals Permanently Inflate Virtual Balances; Documented Self-Correction Paths Are Incorrect

Severity: Low Risk

Context: *(No context files were provided by the reviewer)*

Description: Balancer's `removeLiquidityRecovery` (in `VaultExtension`) bypasses all pool hooks, including `onBeforeRemoveLiquidity`. The `ReClammPool` relies on this hook (`ReClammPool.sol:341–373`) to scale `_lastVirtualBalanceA` and `_lastVirtualBalanceB` proportionally with BPT burns:

```
// ReClammPool.sol:357–367
uint256 poolTotalSupply = _vault.totalSupply(pool);
uint256 bptDelta = poolTotalSupply - exactBptAmountIn;

(uint256 currentVirtualBalanceA, uint256 currentVirtualBalanceB, ) =
→ _computeCurrentVirtualBalances(
    balancesScaled18
);

currentVirtualBalanceA = (currentVirtualBalanceA * bptDelta) / poolTotalSupply;
currentVirtualBalanceB = (currentVirtualBalanceB * bptDelta) / poolTotalSupply;
```

After a Recovery Mode proportional withdrawal, real balances shrink (the Vault handles this) while virtual balances remain at their pre-withdrawal magnitude in storage. This leaves the pool with inflated virtual depth: the invariant $L = (Ra + Va)(Rb + Vb)$ uses virtual balances that are disproportionately large relative to the remaining real balances, producing a flatter swap curve that systematically misprices swaps in favor of swappers and against LPs.

The documentation at `IReClammPool.sol:76–82` states:

“the pool would operate post-recovery with inflated virtual depth until the price drifts out of range (triggering a VB shift) or governance forces recalibration by calling `setDailyPriceShiftExponent` or `setCenterednessMargin`”.

This is analytically incorrect for the proportional-withdrawal case. Here is why none of the listed paths correct the inflated virtual balances:

1. Automatic VB shift via out-of-range detection does not trigger..

Centeredness is computed as $(Ra * Vb) / (Rb * Va)$ (or its inverse, whichever is ≤ 1) at `ReClammMath.sol:595–596`. A proportional recovery withdrawal scales both `Ra` and `Rb` by the same factor `k` (e.g., `k = 0.5` for a 50% withdrawal). Since virtual balances `Va` and `Vb` remain unchanged:

```
centeredness_after = (k * Ra * Vb) / (Va * k * Rb) = (Ra * Vb) / (Va * Rb) =
→ centeredness_before
```

The `k` factors cancel. Centeredness is unchanged. For a pool that was in range before Recovery Mode (`centeredness ≥ margin`), it remains in range after. The condition `centeredness < centerednessMargin` at `ReClammMath.sol:387` never fires, so `computeVirtualBalancesUpdatingPriceRange` is never called and virtual balances are never corrected.

2. `setDailyPriceShiftExponent` and `setCenterednessMargin` do not recalibrate..

Both call `_updateVirtualBalances()` → `_getRealAndVirtualBalances()` → `_computeCurrentVirtualBalances()`, which calls `computeCurrentVirtualBalances`. This function only modifies VBs in two cases:

- Price ratio is updating (not relevant here - Recovery Mode doesn't trigger a price ratio update).
- Pool is out of range (refuted above - centeredness is preserved).

Since neither condition holds, `_updateVirtualBalances` returns the stored (inflated) VBs unchanged.

3. `disableRecoveryMode` does not reset VBs..

`VaultAdmin.disableRecoveryMode` calls `_syncPoolBalancesAfterRecoveryMode`, which updates `_poolTokenBalances` in the Vault's storage to reflect current raw balances. It does not call any pool hook or otherwise touch `_lastVirtualBalanceA` / `_lastVirtualBalanceB`.

The net result is that after a proportional Recovery Mode withdrawal, the pool operates with inflated virtual depth indefinitely. Every subsequent swap is mispriced: the flatter curve gives swappers more output than the real liquidity justifies, with LPs absorbing the difference. The degree of mispricing depends on the withdrawal fraction and the ratio of real to virtual balances in the specific pool, so it cannot be expressed as a single fixed percentage.

The only actual correction path is for governance to call `startPriceRatioUpdate` with a corrective target that forces a VB recalculation, which is not mentioned in the documentation.

Proof of Concept: The core argument is algebraic and does not require a Foundry proof of concept to verify:

1. **Centeredness invariance under proportional withdrawal.** Given $\text{centeredness} = \min((R_a * V_b) / (V_a * R_b), (V_a * R_b) / (R_a * V_b))$, a proportional withdrawal multiplies both R_a and R_b by the same scalar $k \in (0, 1)$. The scalar cancels in both numerator and denominator. Centeredness is unchanged.
2. **VB update short-circuit.** `computeCurrentVirtualBalances` at `ReClammMath.sol:375–390` only modifies VBs if (a) the price ratio is actively updating, or (b) $\text{centeredness} < \text{margin}$. After a proportional recovery withdrawal, neither condition holds. The function returns the stored (inflated) VBs.
3. **Inflated swap curve.** With $V_{a_stored} > V_{a_correct}$ and $V_{b_stored} > V_{b_correct}$, the invariant $L = (R_a + V_a)(R_b + V_b)$ is larger than it should be for the current real balances. In `computeOutGivenIn`:

$$[] \text{ amountOut} = (B_o + V_o) * A_i / (B_i + V_i + A_i)$$

Inflated V_o and V_i produce a larger `amountOut` than justified by the real liquidity, transferring value from LPs to swappers.

To confirm the centeredness invariance concretely: consider a pool with $R_a = 100$, $R_b = 200$, $V_a = 50$, $V_b = 100$. $\text{Centeredness} = (100 * 100) / (50 * 200) = 1.0$. After a 50% recovery withdrawal: $R_a = 50$, $R_b = 100$, $V_a = 50$ (unchanged), $V_b = 100$ (unchanged). $\text{Centeredness} = (50 * 100) / (50 * 100) = 1.0$. Still 1.0, still above any margin (max 90%). The range-shift never triggers.

Recommendation:

1. **Correct the documentation** at `IReClammPool.sol:76–82`. The current text claims governance can recalibrate via `setDailyPriceShiftExponent` or `setCenterednessMargin`, which is analytically incorrect for the proportional-withdrawal scenario. The documentation should state that the only effective correction path is `startPriceRatioUpdate` with a target that forces VB recomputation, and explain why the other setters do not work (centeredness is preserved under proportional withdrawals).
2. **Consider adding a dedicated `recalibrateVirtualBalances()` governance function** (with `onlyWhenVaultIsLocked`) that resets virtual balances from current live balances, specifically for post-recovery correction. This would give governance a direct, documented tool rather than requiring them to reverse-engineer a corrective price ratio target.

Balancer: Fixed in [PR 192](#).

Cantina Managed: [PR 192](#) corrects the inaccurate documentation in `IReClammPool.sol` that claimed governance could recalibrate virtual balances via `setDailyPriceShiftExponent` or `setCenterednessMargin`, and adds a note reiterating that disabling recovery mode after a proportional withdrawal is problematic. No `recalibrateVirtualBalances()` function was added, as pools entering recovery mode are not expected to resume normal operation.

3.2 Informational

3.2.1 Price-ratio update setters are inconsistent with the lock model used by comparable admin setters

Severity: Informational

Context: [ReClammPool.sol#L575-L653](#)

Description: `startPriceRatioUpdate()` and `stopPriceRatioUpdate()` both call `_updateVirtualBalances()` before they modify price-ratio state, and `_updateVirtualBalances()` reads current live balances from the Vault. Other admin setters that follow the same pattern, namely `setDailyPriceShiftExponent()` and `setCenterednessMargin()`, require `onlyWhenVaultIsLocked()`. The comment above `_setDailyPriceShiftExponentAndUpdateVirtualBalances()` also says directly that using pool balances to update virtual balances is dangerous when the Vault is unlocked because those balances are manipulable.

Recommendation: Consider making the safety model consistent across all admin paths that derive state from live balances.

One approach could be to add `onlyWhenVaultIsLocked()` to `startPriceRatioUpdate()` and `stopPriceRatioUpdate()` so they match `setDailyPriceShiftExponent()` and `setCenterednessMargin()`. That removes the direct unlocked-callback variant and makes the implementation easier to reason about.

Balancer: Fixed in [PR 182](#).

Cantina Managed: Fix verified.

3.2.2 Long-idle virtual-balance decay can brick a ReClamm pool after near depletion

Severity: Informational

Context: [ReClammMath.sol#L536-L540](#)

Description: ReClamm updates virtual balances lazily, so elapsed time is only applied when an interaction touches the pool. When the pool drifts far enough out of range that one token side is nearly empty, that depleted side becomes the overvalued side in the price-range update logic. Its virtual balance is decayed multiplicatively over elapsed time against a floor derived from the corresponding real balance. Once that real balance is zero, or close enough to round to the same effect, the floor provides no protection, and a long idle period can decay the overvalued virtual balance to zero inside the update path.

The failure typically unfolds across two interactions. The first, after a long idle period, commits a much smaller virtual balance to storage while still succeeding. A 1-unit `swapSingleTokenExactIn()` was sufficient in the reproduced path, returning zero output while persisting the degraded state. After a second idle period, the next refresh starts from that weakened state and reverts with a division-by-zero panic inside `computeVirtualBalancesUpdatingPriceRange()`. At that point the pool can no longer process swaps, and administrative and proportional-liquidity operations that go through the same recomputation are likely blocked as well.

The reproduced path used the test harness default of 100% daily price shift exponent, the maximum allowed value. That rate drives the virtual balance down far faster than any realistic deployment would configure. At a more typical value (e.g. 10-20% daily), reaching the same degraded state would require the pool to stay nearly depleted and completely idle for a proportionally much longer period. Combined with the prerequisite of near-complete one-sided depletion, this makes the scenario largely theoretical in practice. Nonetheless, the revert path is real: once the state is reached, the pool is bricked regardless of how it got there.

Proof of Concept:

```

// SPDX-License-Identifier: GPL-3.0-or-later

pragma solidity ^0.8.24;

import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import { ReClammPool } from "../../contracts/ReClammPool.sol";

import { BaseReClammTest } from "../../utils/BaseReClammTest.sol";

contract ReClammLongIdleMinimalCaseTest is BaseReClammTest {
    IERC20 internal tokenA;
    IERC20 internal tokenB;

    function setUp() public override {
        poolInitAmount = 1e12;
        super.setUp();

        tokenA = daiIdx == 0 ? IERC20(address(dai)) : IERC20(address(usdc));
        tokenB = daiIdx == 0 ? IERC20(address(usdc)) : IERC20(address(dai));
    }

    function testMinimalUserOnlyCase() public {
        (, , uint256[] memory balancesBeforeDrain, ) = vault.getPoolTokenInfo(pool);

        vm.prank(alice);
        router.swapSingleTokenExactOut(pool, tokenB, tokenA, balancesBeforeDrain[0],
            ↪ MAX_UINT256, MAX_UINT256, false, bytes(""));

        vm.warp(block.timestamp + 11 days + 22 hours);

        vm.prank(alice);
        router.swapSingleTokenExactIn(pool, tokenB, tokenA, 1, 0, MAX_UINT256, false,
            ↪ bytes(""));

        (uint256 virtualBalanceAAfterFirstPoke, uint256 virtualBalanceBAfterFirstPoke) =
            ↪ ReClammPool(pool)
                .getLastVirtualBalances();

        emit log_named_uint("virtual_balance_a_after_first_poke",
            ↪ virtualBalanceAAfterFirstPoke);
        emit log_named_uint("virtual_balance_b_after_first_poke",
            ↪ virtualBalanceBAfterFirstPoke);

        vm.warp(block.timestamp + 1 hours);

        vm.expectRevert();
        ReClammPool(pool).computeCurrentVirtualBalances();

        vm.prank(alice);
        vm.expectRevert();
        router.swapSingleTokenExactIn(pool, tokenB, tokenA, 1, 0, MAX_UINT256, false,
            ↪ bytes(""));
    }
}

```

Recommendation: Consider deriving the minimum for `virtualBalanceOvervalued` from the denominator in the undervalued-side recomputation. That denominator, $(\text{sqrtPriceRatio} - 1) * \text{virtualBalanceOvervalued} - \text{balancesScaledOvervalued}$, must stay strictly positive after fixed-point rounding, which gives a lower bound of $\text{ceil}(\text{balancesScaledOvervalued} / (\text{sqrtScaled18}(\text{sqrtPriceRatio}) - \text{FixedPoint.ONE}))$. Switching the existing `divDown` to `divUp` and adding a `FixedPoint.ONE` guard covers both the zero real-balance case and the zero-denominator case:

```

- virtualBalanceOvervalued = Math.max(
-     virtualBalanceOvervalued,

```

```

-     balancesScaledOvervalued.divDown(sqrtScaled18(sqrtPriceRatio) - FixedPoint.ONE)
- );
+ uint256 minVirtualBalance = Math.max(
+     FixedPoint.ONE,
+     balancesScaledOvervalued.divUp(sqrtScaled18(sqrtPriceRatio) - FixedPoint.ONE)
+ );
+ virtualBalanceOvervalued = Math.max(virtualBalanceOvervalued, minVirtualBalance);

```

A flat clamp to `1e18` avoided the failure in the tested configurations, but it is an empirical observation rather than a derived bound and may not hold across all pool configurations or price ratios. The exact derivation above is preferred. A post-update sanity check before committing refreshed balances to storage would provide additional defense in depth.

Balancer: Fixed in [PR 177](#).

Cantina Managed: [PR 177](#) resolves this by restructuring the virtual balance update so it no longer depends on computing the price range first. `computePriceRange` now computes the max price in a more numerically stable manner, removing the division-by-zero path that previously bricked the pool.

3.2.3 Uninitialized price-ratio getters revert on deployed but uninitialized pools

Severity: Informational

Context: [ReClammPool.sol#L472-L480](#)

Description: `computeCurrentPriceRatio()` and `computeCurrentFourthRootPriceRatio()` revert when called on pools that have been deployed and registered but not yet initialized. Both getters unconditionally derive the current price ratio from live balances and virtual balances, even though virtual balances are still zero in the pre-initialization state, which causes the price-range calculation to hit a division-by-zero path.

This leaves the standalone price-ratio getters inconsistent with adjacent read paths that already handle uninitialized pools more gracefully. In particular, `getReClammPoolDynamicData()` avoids computing the ratio before initialization, while `computeCurrentPriceRange()` returns the configured initial bounds instead of reverting.

If the intended semantics are that no current ratio exists before initialization, an explicit `PoolNotInitialized()` revert would communicate that behavior more clearly than a low-level arithmetic failure. Otherwise, returning the ratio implied by the configured initial price bounds would provide a smoother read path for integrations querying newly created pools.

Proof of Concept:

`test/foundry/ReClammPool.t.sol: testPriceRatioGettersRevertBeforeInitialized()`

```

function testPriceRatioGettersRevertBeforeInitialized() public {
    IERC20[] memory sortedTokens = InputHelpers.sortTokens(tokens);

    (address uninitializedPool, ) = _createPool(
        [address(sortedTokens[a]), address(sortedTokens[b])].toMemoryArray(),
        "BeforeInitTest"
    );

    assertFalse(vault.isPoolInitialized(uninitializedPool), "Pool is initialized");

    (uint256 virtualBalanceA, uint256 virtualBalanceB) =
        ↪ ReClammPool(uninitializedPool).getLastVirtualBalances();
    assertEquals(virtualBalanceA, 0, "Virtual balance A should be zero before
        ↪ initialization");
    assertEquals(virtualBalanceB, 0, "Virtual balance B should be zero before
        ↪ initialization");

    vm.expectRevert();
    ReClammPool(uninitializedPool).computeCurrentPriceRatio();

    vm.expectRevert();

```

```
ReClammPool(uninitializedPool).computeCurrentFourthRootPriceRatio();  
}
```

Recommendation: Consider aligning these getters with the intended pre-initialization API semantics. One approach could be to return the ratio implied by the configured initial price bounds; alternatively, if these getters are meant to be unavailable before initialization, consider reverting explicitly with `PoolNotInitialized()` instead of relying on a low-level arithmetic revert.

Balancer: Fixed in PR 174.

Cantina Managed: PR 174 aligns all "current" getters to revert before initialization: `computeCurrentPriceRatio()`, `computeCurrentFourthRootPriceRatio()`, `computeCurrentVirtualBalances()`, `computeCurrentPoolCenteredness()`, and `isPoolWithinTargetRange()` now revert instead of returning misleading zero or false values. `getReClammPoolDynamicData()` and `computeCurrentPriceRange()` are intentionally left unchanged, as both return meaningful data in the pre-initialization state.

3.2.4 Centeredness boundary is described inconsistently across docs and implementation

Severity: Informational

Context: [ReClammMath.sol#L551-L569](#)

Description: The math documentation describes the pool as IN RANGE when `centeredness > margin` and OUT OF RANGE when `centeredness <= margin`. The implementation uses `centeredness >= margin` as the in-range condition instead, and the tests appear to follow the implementation behavior.

This looks like a documentation mismatch rather than a protocol flaw, however, it is worth making the docs match the real rule precisely.

Recommendation: Consider updating the docs so the published boundary condition matches the implemented one exactly. If the strict `>` wording was intentional, consider revisiting the implementation and tests so the rule is consistent in every layer.

Balancer: Fixed in PR 173.

Cantina Managed: Fix verified.

3.2.5 Factory accepts invalid initial dynamic params that cause initialization to revert

Severity: Informational

Context: [ReClammPool.sol#L132-L163](#)

Description: `ReClammPoolFactory.create()` accepts `dailyPriceShiftExponent` and `centerednessMargin` without checking them against the bounds the pool enforces at run-time. The factory deploys and registers the pool successfully, but the values are stored as immutable initialization parameters and only validated later, inside `onBeforeInitialize()`, when `_setDailyPriceShiftExponent()` and `_setCenterednessMargin()` are called. Passing an out-of-range value produces a pool that is registered in the vault but permanently uninitializable: any attempt to initialize it will revert with `InvalidCenterednessMargin()` or the equivalent error for the shift exponent.

The practical impact is limited to wasted gas and a stuck registry entry. The revert is atomic at initialization time and does not affect any already-live pool. That said, a permissionless factory is a natural place to reject values the pool will deterministically reject anyway, and doing so at `create()` time is strictly cheaper for the caller.

Proof of Concept: `test/poc/ReClammInvalidInitialDynamicParamsPoC.t.sol: testFactoryRegistersPoolWithInvalidInitialCenterednessMarginButInitializeReverts`

```
// SPDX-License-Identifier: GPL-3.0-or-later  
  
pragma solidity ^0.8.24;  
  
import { IERC20 } from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
```

```

import { PoolRoleAccounts } from
↳ "@balancer-labs/v3-interfaces/contracts/vault/VaultTypes.sol";
import { CastingHelpers } from
↳ "@balancer-labs/v3-solidity-utils/contracts/helpers/CastingHelpers.sol";
import { InputHelpers } from
↳ "@balancer-labs/v3-solidity-utils/contracts/helpers/InputHelpers.sol";
import { ArrayHelpers } from
↳ "@balancer-labs/v3-solidity-utils/contracts/test/ArrayHelpers.sol";

import { BaseReClammTest } from "../foundry/utils/BaseReClammTest.sol";
import { ReClammPool } from "../../contracts/ReClammPool.sol";
import { ReClammPoolFactory } from "../../contracts/ReClammPoolFactory.sol";
import { ReClammPriceParams } from "../../contracts/lib/ReClammPoolFactoryLib.sol";
import { IReClammPool } from "../../contracts/interfaces/IReClammPool.sol";

contract ReClammInvalidInitialDynamicParamsPoC is BaseReClammTest {
    using CastingHelpers for address[];
    using ArrayHelpers for *;

    uint256 private constant _INITIAL_AMOUNT = 1_000e18;
    uint256 private constant _INVALID_CENTEREDNESS_MARGIN = 90e16 + 1;

    function testFactoryRegistersPoolWithInvalidInitialCenterednessMarginButInitializeRe
    ↪ verts() public {
        address newPool = _createPoolWithInvalidInitialCenterednessMargin();

        assertTrue(vault.isPoolRegistered(newPool), "factory should register the pool");
        assertFalse(vault.isPoolInitialized(newPool), "pool should start uninitialized");

        IERC20[] memory poolTokens = vault.getPoolTokens(newPool);
        assertEquals(poolTokens.length, 2, "registered pool should expose both tokens");

        uint256[] memory initialBalances =
        ↪ ReClammPool(newPool).computeInitialBalancesRaw(poolTokens[0],
        ↪ _INITIAL_AMOUNT);

        vm.startPrank(lp);
        vm.expectRevert(IReClammPool.InvalidCenterednessMargin.selector);
        router.initialize(newPool, poolTokens, initialBalances, 0, false, bytes(""));
        vm.stopPrank();

        assertTrue(vault.isPoolRegistered(newPool), "failed init should not undo
        ↪ registration");
        assertFalse(vault.isPoolInitialized(newPool), "pool should remain
        ↪ uninitialized");
    }

    function _createPoolWithInvalidInitialCenterednessMargin() internal returns
    ↪ (address) {
        ReClammPoolFactory factoryForPoC =
        ↪ deployReClammPoolFactoryWithDefaultParams(vault);
        IERC20[] memory sortedTokens = InputHelpers.sortTokens([address(usdc),
        ↪ address(dai)].toMemoryArray().asIERC20());
        PoolRoleAccounts memory roleAccounts;

        ReClammPriceParams memory priceParams = ReClammPriceParams({
            initialMinPrice: _initialMinPrice,
            initialMaxPrice: _initialMaxPrice,
            initialTargetPrice: _initialTargetPrice,
            tokenAPriceIncludesRate: _tokenAPriceIncludesRate,
            tokenBPriceIncludesRate: _tokenBPriceIncludesRate
        });

        return
        ↪ factoryForPoC.create(

```

```

        "ReClamm Pool",
        "RECLAMM_POOL",
        vault.buildTokenConfig(sortedTokens),
        roleAccounts,
        _DEFAULT_SWAP_FEE,
        priceParams,
        _DEFAULT_DAILY_PRICE_SHIFT_EXPONENT,
        _INVALID_CENTEREDNESS_MARGIN,
        bytes32(saltNumber++)
    );
}
}

```

Recommendation: Consider mirroring the pool's upper bounds for `dailyPriceShiftExponent` and `centerednessMargin` in `ReClammPoolFactory.create()` or in `ReClammPool`'s constructor, so invalid values revert before deployment and registration rather than after. This is optional fail-early hardening with no protocol-level security impact.

Balancer: Fixed in [PR 172](#).

Cantina Managed: Fix verified.

3.2.6 Target price equal to the upper bound can leave a deployed pool non-initializable

Severity: Informational

Context: [ReClammPoolFactoryLib.sol#L71-L92](#)

Description: `validatePriceParams()` currently accepts configurations where `initialTargetPrice == initialMaxPrice`. That boundary looks valid at deployment time, but the initialization path may still fail when the pool later derives its starting theoretical balances and virtual balances from those immutable price parameters.

The result is a narrow deployment-domain mismatch: the pool can be created successfully, but every initialization attempt can revert for that specific configuration. The impact is limited to the affected pool instance and depends on a deployer-chosen parameter set, so this is better treated as an informational setup hazard than a general protocol bug.

Recommendation: Consider rejecting `initialTargetPrice == initialMaxPrice` during validation unless the initialization path is updated to support that equality case explicitly.

```

- params.initialTargetPrice > params.initialMaxPrice ||
+ params.initialTargetPrice >= params.initialMaxPrice ||

```

Balancer: Fixed in [PR 176](#).

Cantina Managed: Fix verified.

3.2.7 Price-ratio update speed check uses a linear approximation

Severity: Informational

Context: [ReClammPool.sol#L623-L638](#)

Description: `startPriceRatioUpdate()` checks the allowed daily price-ratio update speed with a linear expression based on the ratio delta and the scheduled duration. The live update path does not use that model. At runtime, the pool interpolates the fourth-root price ratio geometrically, so the effective one-day rate is multiplicative rather than linear.

That means the setter is not enforcing the same rule the runtime math actually follows. For updates scheduled between one and two days, the current guard can allow a schedule whose true multiplicative daily change is slightly above the intended cap. For longer updates, the same guard can reject schedules that remain within the runtime-equivalent bound.

This looks more like an intentional simplification than a plain math mistake. The current check is cheap to compute, and it sits in an admin setter rather than a swap path. The problem is that the code and docs do

not present it as an approximation. As written, it reads like the exact policy check, even though it is only a rough substitute for the geometric model used elsewhere.

Two examples show the mismatch:

1. A schedule from 4 to 8.8 over 27 hours passes the current 2x/day check, but its true multiplicative daily rate is about 2.015x/day.
2. A schedule from 2 to 14 over 3 days is rejected by the current check, even though its true multiplicative daily rate is about 1.913x/day.

This is best treated as an informational spec-alignment issue. It does not create a permissionless exploit path, and the gap is fairly small under the protocol's current parameter bounds. Still, the current implementation may not fully enforce the policy that operators would reasonably infer from the runtime math and comments.

Recommendation: Consider picking one of two directions and documenting it clearly.

One approach could be to replace the current linear check with the same multiplicative model the pool uses at runtime. Since this logic only runs in an admin scheduling path, the extra cost of `powUp()` is unlikely to matter much.

If the simpler check is preferred, consider documenting it explicitly as a heuristic rather than the exact runtime-equivalent daily-rate bound. In that case, it would also help to note that the guard is slightly permissive for one-to-two-day schedules and slightly conservative for longer ones.

Balancer: Fixed in [PR 179](#).

Cantina Managed: Fix verified.

3.2.8 Reverting Rate Provider Traps Pool in Recovery Mode Indefinitely

Severity: Informational

Context: (No context files were provided by the reviewer)

Note: This finding concerns pre-existing behavior in the already deployed Balancer Vault contracts rather than ReCLAMM-specific logic. As such, it should be considered out of scope for this review and is not remediable through changes to ReCLAMM itself.

Description: All state-mutating operations in `ReClammPool` depend on reading live balances from the Vault, which internally applies token rates via the registered rate provider. When a rate provider reverts (paused lending protocol, deprecated oracle, self-destructed contract), this revert propagates unhandled through every pool operation.

The dependency chain is:

```
_updateVirtualBalances() // ReClammPool.sol:818
→ _getRealAndVirtualBalances() // ReClammPool.sol:823
→ _vault.getCurrentLiveBalances(address(this)) // ReClammPool.sol:438
→ [Vault internals: _loadPoolData → PoolDataLib.getTokenRate →
→ rateProvider.getRate()]
```

Every governance function calls `_updateVirtualBalances()` as its first meaningful operation:

- `startPriceRatioUpdate` - line 604: `_updateVirtualBalances()`.
- `stopPriceRatioUpdate` - line 643: `_updateVirtualBalances()`.
- `setDailyPriceShiftExponent` - line 666: `_setDailyPriceShiftExponentAndUpdateVirtualBalances() → _updateVirtualBalances()`.
- `setCenterednessMargin` - line 673: `_updateVirtualBalances()`.

For `onSwap`, the Vault itself applies rates when preparing `request.balancesScaled18` before calling the pool, so a reverting rate provider also blocks all swaps.

The standard Balancer response to a broken rate provider is to enable Recovery Mode, which allows proportional LP withdrawals without requiring rate computation. This works correctly - LPs can exit.

The problem is the **exit path from Recovery Mode**. `VaultAdmin.disableRecoveryMode` calls `_syncPoolBalancesAfterRecoveryMode`, which internally calls `_loadPoolData`, which calls `getTokenRate` on the rate provider. If the rate provider is still reverting at that point, `disableRecoveryMode` itself reverts. This creates a circular dependency:

1. Rate provider breaks → all swaps and governance functions revert.
2. Governance enables Recovery Mode → LP withdrawals work (this path does not read rates).
3. Rate provider is still broken.
4. Governance calls `disableRecoveryMode` → reverts because `_syncPoolBalancesAfterRecoveryMode` reads rates.
5. The pool is trapped in Recovery Mode until the rate provider recovers.

During the entire outage:

- No swaps can execute.
- No governance parameter changes can be made (`startPriceRatioUpdate`, `stopPriceRatioUpdate`, `setDailyPriceShiftExponent`, `setCenterednessMargin` all revert).
- Recovery Mode cannot be disabled.
- Duration is entirely dependent on the external rate provider - it is unbounded from the pool's perspective.

This is specific to pools with rate providers (e.g., `wstETH`, `waUSDC`). Pools using only standard tokens (`TokenType.STANDARD`) are unaffected since `getTokenRate` returns `FixedPoint.ONE` without an external call.

Historical precedent exists for rate provider outages: Lido has paused `stETH` operations, and Aave has had governance-triggered pauses on lending markets that back rate-providing wrapped tokens.

The Vault's own documentation at `VaultAdmin.sol:396` acknowledges the risk:

"Live balances cannot be updated in Recovery Mode, as this would require making external calls to update rates, which could fail".

This comment recognizes that rate calls can fail during Recovery Mode, but does not address the consequence that `disableRecoveryMode` itself depends on the same rate calls, creating the exit trap.

Proof of Concept: The dependency chain can be verified by tracing the code paths:

1. **All governance functions depend on `_vault.getCurrentLiveBalances`.** Each of the four setters calls `_updateVirtualBalances()` (line 818), which calls `_getRealAndVirtualBalances()` (line 823), which calls `_vault.getCurrentLiveBalances(address(this))` (line 438). This Vault call applies rates internally. If `rateProvider.getRate()` reverts, the entire call reverts.
2. **`disableRecoveryMode` depends on rate providers.** `VaultAdmin.disableRecoveryMode` → `_syncPoolBalancesAfterRecoveryMode` → `_loadPoolData` → `PoolDataLib.getTokenRate` → `rateProvider.getRate()`. If the rate provider reverts, `disableRecoveryMode` reverts.
3. **Recovery Mode withdrawals do NOT depend on rate providers.** `removeLiquidityRecovery` in `VaultExtension` uses stored raw balances and bypasses hooks and rate computations. This is why LP exits work even with a broken rate provider.
4. **The trap.** Steps 1-3 together mean: once a rate provider breaks, governance can enter Recovery Mode (enabling LP exits) but cannot exit Recovery Mode until the rate provider recovers. The pool is fully inoperable for an unbounded duration.

Recommendation: This is fundamentally an upstream Balancer Vault concern, since the rate provider call chain originates in `PoolDataLib.getTokenRate`. The most robust fix would be at that level:

1. **Upstream (preferred):** Wrap rate provider calls in try/catch within `PoolDataLib.getTokenRate`, falling back to the last known rate on failure. This would protect all pool types, not just `ReClammPool`.

2. **Pool-level mitigation:** If an upstream fix is not feasible, consider implementing a circuit breaker at the pool level - for example, caching the last successful rate and using the cached value when the rate provider reverts. This would allow governance operations and `disableRecoveryMode` to proceed with stale-but-safe rates.
3. **At minimum:** Document the Recovery Mode exit dependency on rate provider liveness, so that governance operators understand that enabling Recovery Mode during a rate provider outage is a one-way door until the provider recovers.

Balancer: Acknowledged.

Cantina Managed: Acknowledged.

3.2.9 `toDailyPriceShiftBase` Silently Disables Price-Shifting for Sub-Threshold Inputs; NatSpec Formula Inverted

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: Two independent issues in the daily price shift parameter handling.

Issue 1 - Silent zero-shift for sub-threshold exponents.

`toDailyPriceShiftBase` at `ReClammMath.sol:731–732`:

```
function toDailyPriceShiftBase(uint256 dailyPriceShiftExponent) internal pure returns
↳ (uint256) {
    return FixedPoint.ONE - dailyPriceShiftExponent /
    ↳ _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT;
}
```

`_PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT` is 124649, a raw integer (not 18-decimal FP). The division `dailyPriceShiftExponent / 124649` is Solidity integer division. For any `dailyPriceShiftExponent < 124649`, the result truncates to 0, and the function returns `FixedPoint.ONE - 0 = 1e18`.

When `dailyPriceShiftBase = 1e18`, the price range tracking in `computeVirtualBalancesUpdating PriceRange` is completely disabled:

```
// ReClammMath.sol:532–534
virtualBalanceOvervalued = virtualBalanceOvervalued.mulDown(
    dailyPriceShiftBase.powDown(duration * FixedPoint.ONE)
);
```

`powDown(1e18, anything) = 1e18`, so `mulDown(VB, 1e18) = VB` - the virtual balance never changes regardless of how far out of range the pool drifts.

The round-trip through `_setDailyPriceShiftExponent` at `ReClammPool.sol:784–787` confirms the silent failure:

```
uint256 dailyPriceShiftBase = dailyPriceShiftExponent.toDailyPriceShiftBase(); // → 1e18
dailyPriceShiftExponent = dailyPriceShiftBase.toDailyPriceShiftExponent(); // →
↳ (1e18 - 1e18) * 124649 = 0
```

The emitted `DailyPriceShiftExponentUpdated` event reports `dailyPriceShiftExponent = 0` and `dailyPriceShiftBase = 1e18`. A governance actor setting a small but non-zero exponent would see it silently rounded to zero with no revert. The threshold (`124649 wei = 1.24649e-13` as 18-decimal FP) is extremely small, so this requires a near-zero input to trigger. In practice, no governance actor would intentionally set such a tiny value, but the silent failure mode means there is no feedback that the value was rejected.

Issue 2 - Inverted NatSpec formula.

The NatSpec for `getDailyPriceShiftBase` at `IReClammPool.sol:346–347`:

```
/// @dev Equals dailyPriceShiftExponent / _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT.
```

The actual formula at `ReClammMath.sol:732`:

```
return FixedPoint.ONE - dailyPriceShiftExponent /
↳ _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT;
```

The `1 -` prefix is missing from the documentation. The documented formula computes `dailyPriceShiftExponent / 124649`, which is the tau value, not the `base = 1 - tau` value. Any off-chain integration relying on the documented formula to reconstruct `dailyPriceShiftBase` from `dailyPriceShiftExponent` would compute the wrong result - off by approximately $1e18$.

Recommendation: Issue 1: Add a minimum threshold check, or document that exponents below 124649 wei are silently rounded to zero. If a lower bound is preferred:

```
function toDailyPriceShiftBase(uint256 dailyPriceShiftExponent) internal pure returns
↳ (uint256) {
    if (dailyPriceShiftExponent != 0 && dailyPriceShiftExponent <
↳ _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT) {
        revert DailyPriceShiftExponentTooLow();
    }
    return FixedPoint.ONE - dailyPriceShiftExponent /
↳ _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT;
}
```

Issue 2: Correct the NatSpec at `IReClammPool.sol:347`:

```
- * @dev Equals dailyPriceShiftExponent / _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT.
+ * @dev Equals 1 - dailyPriceShiftExponent / _PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT.
```

Balancer: Fixed in PR 190.

Cantina Managed: Fix verified.

3.2.10 Per-Block Virtual Balance Freeze Creates First-Mover MEV Surface

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: `computeCurrentVirtualBalances` short-circuits when `lastTimestamp == block.timestamp`, returning stored virtual balances without recomputation:

```
// ReClammMath.sol:338-344
uint32 currentTimestamp = block.timestamp.toUint32();

if (lastTimestamp == currentTimestamp) {
    return (lastVirtualBalanceA, lastVirtualBalanceB, false);
}
```

When the pool is out of range (`centeredness < margin`), virtual balances change over time. The overvalued side's VB decays per `computeVirtualBalancesUpdatingPriceRange` at `ReClammMath.sol:532-534`:

```
virtualBalanceOvervalued = virtualBalanceOvervalued.mulDown(
    dailyPriceShiftBase.powDown(duration * FixedPoint.ONE)
);
```

where `duration = currentTimestamp - lastTimestamp` (capped at 30 days) and `dailyPriceShiftBase = 1 - dailyPriceShiftExponent / 124649`.

The first pool interaction in a new block triggers this VB recomputation. `onSwap` at `ReClammPool.sol:191-200` computes the new VBs, stores them via `_setLastVirtualBalances`, and updates `_lastTimestamp` to `block.timestamp`. All subsequent interactions in the same block hit the `lastTimestamp == currentTimestamp` short-circuit and use the frozen VBs.

This means the first interactor in each block unilaterally determines the VB state for all same-block users. When the pool is out of range, this creates a per-block MEV surface: a searcher can front-run other transactions to be the first interactor, capturing the VB shift that accumulated since the previous block.

Deriving the per-block VB decay.

The internal constant `_PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT = 124649` is derived from the constraint that at 100% daily shift, the price range doubles in one day. From the code comments at `ReClammMath.sol:33-46`:

```
tau = 1 - pow(2, 1/86401)
ADJUSTMENT = 100% / tau = 124649
```

The `dailyPriceShiftBase` is computed as:

```
base = 1 - dailyPriceShiftExponent / 124649
```

Per block (12 seconds on Ethereum mainnet), the overvalued VB decays by:

```
Vo_new = Vo * base^12
```

At various configurations:

Daily Shift	base	base^12	Per-block Vo decay
100%	0.999991977	0.999903734	0.00963%
50%	0.999995989	0.999951866	0.00481%
10%	0.999999198	0.999990373	0.00096%

Verification: at 100%, $\text{base}^{86400} \approx 0.5$, confirming the overvalued VB halves in one day, which corresponds to a price doubling as designed.

Profitability analysis.

The maximum per-block extraction is bounded by the VB decay applied to the pool's virtual depth. A round-trip extraction (swap in one direction, then reverse) incurs swap fees twice. At the minimum swap fee of 0.001% (`_MIN_SWAP_FEE_PERCENTAGE = 0.001e16`), the round-trip fee cost is $2 * 0.001\% = 0.002\%$ of notional.

Daily Shift	Pool Size	Gross Extraction	Round-trip Fee (min)	Net (before gas)
100%	1M	96.27	20.00	76.27
100%	100K	9.63	2.00	7.63
50%	1M	48.13	20.00	28.13
50%	100K	4.81	2.00	2.81
10%	1M	9.63	20.00	-10.37
10%	100K	0.96	2.00	-1.04

The break-even point where gross extraction equals round-trip fees (at minimum fee) is at approximately **21% daily shift exponent**. Below this, the MEV is not profitable at any pool size even before accounting for gas. Above this threshold, profitability depends on pool size and actual swap fee (which may be set higher than the minimum).

At realistic configurations (10-20% daily shift), the per-block VB change is smaller than the minimum round-trip swap fee, making this MEV surface economically irrelevant. At the maximum configuration (100% daily shift), extraction of ~\$76 per block is possible on a \$1M pool, but this is the most aggressive parameter setting.

Proof of Concept: No Foundry proof of concept is needed. The derivation is arithmetic from the constants in the code:

- `_PRICE_SHIFT_EXPONENT_INTERNAL_ADJUSTMENT = 124649` (ReClammMath.sol:46).
- At 100% daily shift: $\text{base} = 1e18 - 1e18 / 124649 = 999991977472743487$.
- Per block (12s): $\text{base}^{12} \approx 0.999903734 \rightarrow$ VB decays by 0.00963%.
- On a 1M pool: $1,000,000 * 0.0000963 = \96.27 gross.

5. Minimum round-trip fee: $\$1,000,000 * 0.001\% * 2 = \20 .

6. Net before gas: $\$96.27 - \$20 = \$76.27$.

7. Break-even daily shift: solving $\text{base}^{12} = 1 - 0.00002$ gives $\sim 21\%$.

At 10% daily shift (a typical operational value), $\$1M * 0.00000963 = \9.63 gross, which is below the \$20 minimum round-trip fee. Not profitable.

Recommendation: The per-block freeze is an intentional design pattern that prevents within-block VB manipulation (without it, multiple interactions within a single block could each trigger VB recomputation with different effective timestamps, creating a worse manipulation surface).

Consider adding NatSpec documentation to `computeCurrentVirtualBalances` explicitly noting the per-block VB freeze as a known design tradeoff with a bounded MEV surface, and that at typical operational configurations (daily shift exponent $\leq 20\%$), the per-block VB change is smaller than the minimum round-trip swap fee.

Balancer: Fixed in [PR 189](#).

Cantina Managed: Fix verified.

3.2.11 Unused `_MAX_TOKEN_DECIMALS` constant

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: `_MAX_TOKEN_DECIMALS = 18` is defined independently in both `ReClammPool.sol:60` and `ReClammPoolHelper.sol:27`. The copy in `ReClammPool.sol` is unused, it is never referenced anywhere in the contract. The copy in `ReClammPoolHelper.sol` is actively used for decimal scaling at lines 61 and 75.

Recommendation: Remove `_MAX_TOKEN_DECIMALS` from `contracts/ReClammPool.sol:60`.

Balancer: Fixed in [PR 187](#).

Cantina Managed: Fix verified.

3.2.12 `PriceRatioDeltaBelowMin` Error Parameter and NatSpec Use Wrong Domain Label

Severity: Informational

Context: (No context files were provided by the reviewer)

Description: The error `PriceRatioDeltaBelowMin` at `IReClammPool.sol:228` names its parameter `fourthRootPriceRatioDelta`:

```
error PriceRatioDeltaBelowMin(uint256 fourthRootPriceRatioDelta);
```

The NatSpec for `minPriceRatioDelta` at `IReClammPool.sol:46` also describes it in fourth-root terms:

```
/// @param minPriceRatioDelta The minimum absolute difference between current and new  
↪ fourth root price ratio
```

The implementation at `ReClammPool.sol:612–621` computes and checks the delta in **full price-ratio space**, not fourth-root space:

```
uint256 priceRatioDelta;  
unchecked {  
    priceRatioDelta = endPriceRatio >= startPriceRatio  
        ? endPriceRatio - startPriceRatio  
        : startPriceRatio - endPriceRatio;  
}  
  
if (priceRatioDelta < _MIN_PRICE_RATIO_DELTA) {  
    revert PriceRatioDeltaBelowMin(priceRatioDelta);  
}
```

endPriceRatio and startPriceRatio are full price ratios (maxPrice / minPrice), not fourth roots. The emitted numeric value is correct, it is just labeled with the wrong domain name. No behavioral impact.

Recommendation:

```
// IReClammPool.sol:228
- error PriceRatioDeltaBelowMin(uint256 fourthRootPriceRatioDelta);
+ error PriceRatioDeltaBelowMin(uint256 priceRatioDelta);

// IReClammPool.sol:46
- /// @param minPriceRatioDelta The minimum absolute difference between current and new
  ↪ fourth root price ratio
+ /// @param minPriceRatioDelta The minimum absolute difference between current and new
  ↪ price ratio
```

Balancer: Fixed in PR 188.

Cantina Managed: Fix verified.

3.2.13 Centeredness-preserving price-ratio updates can shift spot price in edge cases

Severity: Informational

Context: (No context files were provided by the reviewer)

Summary: When a price-ratio update is in progress, `computeVirtualBalancesUpdatingPriceRatio()` adjusts virtual balances to preserve the pool's current centeredness rather than its spot price. A trader who swaps after the update is scheduled can shape the centeredness value the update will later preserve, then reverse the trade once the update has shifted the virtual balances and collect the difference. Under aggressive parameters (price ratio doubling over one day with a minimum fee), a 100,000-unit setup swap returned about 1,094 units of net profit over the baseline after one day.

Description: `startPriceRatioUpdate()` allows an authorized account to schedule a gradual change to the pool's price ratio. During swaps and liquidity operations, `computeCurrentVirtualBalances()` applies any in-progress update by calling `computeVirtualBalancesUpdatingPriceRatio()`, which solves new virtual balances that match the interpolated target price ratio while holding centeredness constant.

Centeredness is not fixed at the moment the update is scheduled. It is recomputed from current real balances and the most recent virtual balances each time the update runs. A trader can therefore swap after the update is scheduled, move the live balance ratio away from center, and still remain inside the centeredness margin. The in-progress update then preserves that skewed centeredness as it progresses. Because centeredness and spot price encode different properties of the pool, preserving one does not preserve the other. The update can move the spot price even when no external market price has changed.

To profit from this, a trader must swap after the update is scheduled but before it has made meaningful progress, skewing real balances off center without breaching the centeredness margin. The pool is then left in that skewed state long enough for the update to produce a visible spot shift, after which the trader reverses the trade. The practical limits are the widening size, fee level, pool depth, available capital, and whether arbitrageurs or normal swap flow resets the curve before the reversal.

Impact: Under aggressive widening parameters (price ratio doubling over one day, 0.001% fee), a 100,000-unit setup swap returned about 101,092 units after the reverse leg, versus about 99,998 units in a run without the update - roughly 1,094 units of profit. With a 5% daily price-ratio update instead, meaningful profit required either very large setup trades or long waiting periods. A near-margin setup of about 790,500 units (roughly 79% of the pool's real token-A balance) showed the first positive return after 21 twelve-second blocks, but that margin was only about 0.28 units before gas and competition. Reaching 100 units required about 177 blocks; reaching 1,000 units required about 1,582 blocks. In practice, LPs bear the cost of any spot shift the update produces, and a patient or well-capitalized trader can capture it.

Likelihood: Exploitation requires a valid widening update, a setup swap large enough to push the pool near the centeredness margin without crossing it, enough elapsed blocks for the update to shift spot materially, low fees, and no significant competition during the wait. Under a 5% daily update, the earliest profitable point appeared only when the setup consumed roughly 79% of the pool's real balance - a position that is visible on-chain and leaves the attacker exposed for minutes or hours. Likelihood increases

when updates are large, fees are low, pools are thin, and the update schedule is publicly announced in advance.

Proof of Concept: test/poc/ReClammPriceRatioWideningCenterednessPoC.t.sol:

```
// SPDX-License-Identifier: GPL-3.0-or-later

pragma solidity ^0.8.24;

import { FixedPoint } from
    ↪ "@balancer-labs/v3-solidity-utils/contracts/math/FixedPoint.sol";
import { ArrayHelpers } from
    ↪ "@balancer-labs/v3-solidity-utils/contracts/test/ArrayHelpers.sol";

import { BaseReClammTest } from "../foundry/utils/BaseReClammTest.sol";
import { ReClammPool } from "../../contracts/ReClammPool.sol";
import { ReClammPoolMock } from "../../contracts/test/ReClammPoolMock.sol";
import { a, b } from "../../contracts/lib/ReClammMath.sol";

contract ReClammPriceRatioWideningCenterednessPoC is BaseReClammTest {
    using FixedPoint for uint256;
    using ArrayHelpers for *;

    function setUp() public override {
        setInitializationPrices(1000e18, 4000e18, 2000e18);
        super.setUp();
    }

    function testInRangeWideningUpdateCanMakeRoundTripProfitable() public {
        uint256 startSnapshot = vm.snapshotState();
        uint256 setupAmountA = 100_000e18;
        uint256 widenedPriceRatio = 8e18;

        uint256[] memory scaledBalances = [uint256(_initialBalances[a] * 1000),
        ↪ uint256(_initialBalances[b] * 1000)]
            .toMemoryArray();

        _topUpVaultReservesTo(scaledBalances);
        _configurePoolState(
            scaledBalances,
            _initialVirtualBalances[a] * 1000,
            _initialVirtualBalances[b] * 1000,
            ReClammPool(pool).computeCurrentPriceRatio(),
            _DEFAULT_DAILY_PRICE_SHIFT_EXPONENT,
            _DEFAULT_CENTEREDNESS_MARGIN
        );

        vm.prank(admin);
        ReClammPool(pool).startPriceRatioUpdate(widenedPriceRatio, block.timestamp,
        ↪ block.timestamp + 1 days);

        vm.prank(alice);
        uint256 amountBOut = router.swapSingleTokenExactIn(pool, dai, usdc,
        ↪ setupAmountA, 0, MAX_UINT256, false, "");

        (uint256 centerednessAfterSetup, ) =
        ↪ ReClammPool(pool).computeCurrentPoolCenteredness();
        assertGt(centerednessAfterSetup, _DEFAULT_CENTEREDNESS_MARGIN, "setup must stay
        ↪ in range");
        assertLt(centerednessAfterSetup, 1e18, "setup must move the pool away from
        ↪ center");

        skip(1 days);

        vm.prank(alice);
        uint256 amountAReturnedWithUpdate = router.swapSingleTokenExactIn(
            pool,
```

```

        usdc,
        dai,
        amountBOut,
        0,
        MAX_UINT256,
        false,
        ""
    );

    vm.revertToState(startSnapshot);

    _topUpVaultReservesTo(scaledBalances);
    _configurePoolState(
        scaledBalances,
        _initialVirtualBalances[a] * 1000,
        _initialVirtualBalances[b] * 1000,
        ReClammPool(pool).computeCurrentPriceRatio(),
        DEFAULT_DAILY_PRICE_SHIFT_EXPONENT,
        DEFAULT_CENTEREDNESS_MARGIN
    );

    vm.prank(alice);
    uint256 baselineAmountBOut = router.swapSingleTokenExactIn(
        pool,
        dai,
        usdc,
        setupAmountA,
        0,
        MAX_UINT256,
        false,
        ""
    );

    skip(1 days);

    vm.prank(alice);
    uint256 baselineAmountAReturned = router.swapSingleTokenExactIn(
        pool,
        usdc,
        dai,
        baselineAmountBOut,
        0,
        MAX_UINT256,
        false,
        ""
    );

    emit log_named_uint("amount_b_out", amountBOut);
    emit log_named_uint("with_update_amount_a_back", amountAReturnedWithUpdate);
    emit log_named_uint("baseline_amount_a_back", baselineAmountAReturned);
    emit log_named_int(
        "extra_profit_vs_baseline",
        int256(amountAReturnedWithUpdate) - int256(baselineAmountAReturned)
    );

    assertGt(amountAReturnedWithUpdate, setupAmountA, "widening update should make
    ↪ the round trip profitable");
    assertLt(baselineAmountAReturned, setupAmountA, "without the update the same
    ↪ round trip should not profit");
}

function _configurePoolState(
    uint256[] memory balances,
    uint256 virtualBalanceA,
    uint256 virtualBalanceB,
    uint256 priceRatio,

```

```

    uint256 dailyPriceShiftExponent,
    uint64 centerednessMargin
) internal {
    vault.manualSetPoolBalances(pool, balances, balances);
    ReClammPoolMock(pool).setLastVirtualBalances([virtualBalanceA,
    ↪ virtualBalanceB].toMemoryArray());
    ReClammPoolMock(pool).manualSetCenterednessMargin(centerednessMargin);
    ReClammPoolMock(pool).setLastTimestamp(block.timestamp);
    ReClammPoolMock(pool).manualStartPriceRatioUpdate(priceRatio, block.timestamp,
    ↪ block.timestamp);

    vm.prank(admin);
    ReClammPool(pool).setDailyPriceShiftExponent(dailyPriceShiftExponent);
}

function _topUpVaultReservesTo(uint256[] memory targetBalances) internal {
    if (targetBalances[usdcIdx] > _initialBalances[usdcIdx]) {
        usdc.mint(address(vault), targetBalances[usdcIdx] -
        ↪ _initialBalances[usdcIdx]);
    }

    if (targetBalances[daiIdx] > _initialBalances[daiIdx]) {
        dai.mint(address(vault), targetBalances[daiIdx] - _initialBalances[daiIdx]);
    }

    vault.manualSetReservesOf(usdc, targetBalances[usdcIdx]);
    vault.manualSetReservesOf(dai, targetBalances[daiIdx]);
}
}

```

Observed result:

```

amount_b_out:           194,306,938.447225124514261679
with_update_amount_a_back: 101,092.345969514184272722
baseline_amount_a_back:   99,998.056920347168847494
extra_profit_vs_baseline:  1,094.289049167015425228

```

Recommendation: Consider documenting this behavior in the protocol specification: price-ratio updates preserve centeredness, not spot price, so a widening update applied to a skewed pool will move the spot price. Operators scheduling large or fast updates should prefer doing so when the pool is near center, or should split large updates into smaller increments to limit the size of any single spot shift. Monitoring during active update windows can help identify large setup trades that may indicate an extraction attempt.

A stronger fix is to change `computeVirtualBalancesUpdatingPriceRatio()` to preserve spot price rather than centeredness when solving for new virtual balances. The spot-preserving formulation scales both total balances by a common factor s and solves for s from the quadratic implied by the target price ratio. Letting Q be the square root of the target price ratio, R_a , R_b be current real balances, and $T_a = R_a + V_a$, $T_b = R_b + V_b$ be current total balances:

```

A = Ra / Ta,  B = Rb / Tb

(Q - 1) * s2 - Q * (A + B) * s + Q * A * B = 0

Va' = s * Ta - Ra
Vb' = s * Tb - Rb

```

This removes the spot-shift mechanism, but it means updates may change centeredness as a side effect. If the spot-preserving solve leaves centeredness below the protocol margin, the existing out-of-range repricing path will run and move spot anyway, so very large single-step updates may still need to be split regardless.

Optional spot-preserving implementation diff:

```

diff --git a/contracts/lib/ReClammMath.sol b/contracts/lib/ReClammMath.sol
- -- a/contracts/lib/ReClammMath.sol
+ ++ b/contracts/lib/ReClammMath.sol

```

```

@@
- /**
-  * @notice Compute the virtual balances of the pool when the price ratio is
↪ updating.
-  * @dev This function uses a Bhaskara formula to shrink/expand the price interval
↪ by recalculating the virtual
-  * balances. It'll keep the pool centeredness constant, and track the desired
↪ price ratio. To derive this formula,
-  * we need to solve the following simultaneous equations:
-  *
-  * 1. centeredness = (Ra * Vb) / (Rb * Va)
-  * 2. PriceRatio = invariant^2/(Va * Vb)^2 (maxPrice / minPrice)
-  * 3. invariant = (Va + Ra) * (Vb + Rb)
-  *
-  * Substitute [3] in [2]. Then, isolate one of the V's. Finally, replace the
↪ isolated V in [1]. We get a quadratic
-  * equation that will be solved in this function.
+ /**
+  * @notice Compute the virtual balances of the pool when the price ratio is
↪ updating.
+  * @dev This function uses a Bhaskara formula to shrink/expand the price interval
↪ by recalculating the virtual
+  * balances. It'll keep the current spot price constant, and track the desired
↪ price ratio. Since the spot price is
+  * preserved, the post-update total balances remain on the same ray as the current
↪ total balances:
+  *
+  * 1. Ta = Ra + Va
+  * 2. Tb = Rb + Vb
+  * 3. Ta' = s * Ta
+  * 4. Tb' = s * Tb
+  *
+  * The target square root price ratio Q is:
+  *
+  *  $Q = (Ta' * Tb') / ((Ta' - Ra) * (Tb' - Rb))$ 
+  *
+  * If  $A = Ra / Ta$  and  $B = Rb / Tb$ , substituting [3] and [4] gives:
+  *
+  *  $Q = s^2 / ((s - A) * (s - B))$ 
+  *
+  * Expanding gives the quadratic:
+  *
+  *  $(Q - 1) * s^2 - Q * (A + B) * s + Q * A * B = 0$ 
+  *
+  * The positive root above both real-balance ratios gives the new scale factor.
+  *
+  * @param currentFourthRootPriceRatio The current fourth root of the price ratio
↪ of the pool
+  * @param balancesScaled18 Current pool balances, sorted in token registration
↪ order
@@
function computeVirtualBalancesUpdatingPriceRatio(
    uint256 currentFourthRootPriceRatio,
    uint256[] memory balancesScaled18,
    uint256 lastVirtualBalanceA,
    uint256 lastVirtualBalanceB
) internal pure returns (uint256 virtualBalanceA, uint256 virtualBalanceB) {
-    // Compute the current pool centeredness, which will remain constant.
-    (uint256 poolCenteredness, bool isPoolAboveCenter) = computeCenteredness(
-        balancesScaled18,
-        lastVirtualBalanceA,
-        lastVirtualBalanceB
-    );
-
-    // The overvalued token is the one with a lower token balance (therefore,
↪ rarer and more valuable).

```

```

-      (
-          uint256 balanceTokenUndervalued,
-          uint256 lastVirtualBalanceUndervalued,
-          uint256 lastVirtualBalanceOvervalued
-      ) = isPoolAboveCenter
-          ? (balancesScaled18[a], lastVirtualBalanceA, lastVirtualBalanceB)
-          : (balancesScaled18[b], lastVirtualBalanceB, lastVirtualBalanceA);
-
-      // The centeredness-preserving formula for Vu (Virtual balance undervalued) is
-      ↪ a quadratic equation, with terms:
-      // a = Q0 - 1
-      // b = - Ru (1 + C)
-      // c = - Ru^2 C
-      // where Q0 is the square root of the price ratio, Ru is the undervalued token
-      ↪ balance, and C is the
-      // centeredness. Applying Bhaskara, we'd have: Vu = (-b + sqrt(b^2 - 4ac)) /
-      ↪ 2a.
-      // The Bhaskara expression can be simplified by replacing a, b and c with
-      ↪ these terms, which leads to:
-      // +-----+
-      // |
-      // |          Ru * (1 + C + sqrt(1 + C (C + 4 * Q0 - 2)))
-      // |      Vu = -----
-      // |                      2 * (Q0 - 1)
-      // |
-      // +-----+
-      uint256 sqrtPriceRatio =
-      ↪ currentFourthRootPriceRatio.mulDown(currentFourthRootPriceRatio);
-
-      // Using FixedPoint math as little as possible to improve the precision of the
-      ↪ result.
-      // Note: The input of Math.sqrt must be a 36-decimal number, so that the final
-      ↪ result is 18 decimals.
-      uint256 virtualBalanceUndervalued = (balanceTokenUndervalued *
-      (FixedPoint.ONE +
-      poolCenteredness +
-      Math.sqrt(poolCenteredness * (poolCenteredness + 4 * sqrtPriceRatio -
-      ↪ 2e18) + 1e36))) /
-      (2 * (sqrtPriceRatio - FixedPoint.ONE));
-
-      uint256 virtualBalanceOvervalued = (virtualBalanceUndervalued *
-      ↪ lastVirtualBalanceOvervalued) /
-      lastVirtualBalanceUndervalued;
-
-      (virtualBalanceA, virtualBalanceB) = isPoolAboveCenter
-      ? (virtualBalanceUndervalued, virtualBalanceOvervalued)
-      : (virtualBalanceOvervalued, virtualBalanceUndervalued);
+      uint256 totalBalanceA = balancesScaled18[a] + lastVirtualBalanceA;
+      uint256 totalBalanceB = balancesScaled18[b] + lastVirtualBalanceB;
+
+      uint256 balanceRatioA = balancesScaled18[a].divDown(totalBalanceA);
+      uint256 balanceRatioB = balancesScaled18[b].divDown(totalBalanceB);
+
+      uint256 quadraticA = sqrtPriceRatio - FixedPoint.ONE;
+      uint256 quadraticB = sqrtPriceRatio.mulDown(balanceRatioA + balanceRatioB);
+      uint256 quadraticC =
-      ↪ sqrtPriceRatio.mulDown(balanceRatioA).mulDown(balanceRatioB);
+
+      uint256 discriminant = quadraticB.mulDown(quadraticB) - 4 *
-      ↪ quadraticA.mulDown(quadraticC);
+      uint256 scale = (quadraticB + sqrtScaled18(discriminant)).divUp(2 *
-      ↪ quadraticA);
+
+      virtualBalanceA = totalBalanceA.mulUp(scale) - balancesScaled18[a];
+      virtualBalanceB = totalBalanceB.mulUp(scale) - balancesScaled18[b];

```

```
}
```

Balancer: Acknowledged. It's an interesting alternative, but as stated above there's risk of going out of the target range by following this method. At this point we'd rather not introduce significant changes in the code whenever possible. Price ratio updates are meant to be rather infrequent in any case.

Cantina Managed: Acknowledged.